STL — referência rápida

1 Containers em um relance

Container	Busca	Inserção	Remoção
`vector`	$O(n)$	$O(1)^*$ fim	$O(1)$ fim
`deque`	$O(n)$	$O(1)^*$ pontas	$O(1)$ pontas
`list`	$O(n)$	$O(1)$ c/ it.	$O(1)$ c/ it.
`set`, `map`	$O(\log n)$	$O(\log n)$	$O(\log n)$
`unordered_*`	$O(1)^*$	$O(1)^*$	$O(1)^*$
`priority_queue`	topo $O(1)$	$O(\log n)$	$O(\log n)$
`bitset<N>`	$O(1)$	—	—

* amortizado ou médio. No `unordered_*` o pior caso é $O(n)$ por operação, quando há muitas colisões.

Percorrer qualquer container é $O(n)$, e `.size()` e `.empty()` são $O(1)$ em todos (em `list`, `.size()` é $O(1)$ desde C++11).

2 vector

```
vector<int> v(n, 0);        // n cópias de 0
vector<vector<int>> g(n);   // n listas vazias
vector<vector<ll>> m(n, vector<ll>(m, 0));

v.push_back(x);     // O(1) amortizado
v.pop_back();       // O(1)
v.back(); v.front();// O(1)
v[i];               // O(1), sem checagem
v.size(); v.empty();// O(1)
v.clear();          // O(n), capacidade fica
v.resize(k);        // O(|k - n|)
v.assign(n, x);     // O(n)
v.reserve(n);       // evita realocações
v.insert(it, x);    // O(n) - desloca o resto
v.erase(it);        // O(n)
v.erase(it, it2);   // O(n)
```

Realocar invalida **todos** os iteradores e ponteiros. Guarde índices, não iteradores, se o vetor pode crescer.

Apagar mantendo a ordem (erase–remove, $O(n)$):

```
v.erase(remove(v.begin(), v.end(), x), v.end());
v.erase(remove_if(v.begin(), v.end(),
        [](int a){ return a % 2 == 0; }), v.end());
```

Apagar sem manter a ordem ($O(1)$): `swap(v[i], v.back()); v.pop_back();`

3 Ordenação e busca

```
sort(v.begin(), v.end());            // O(n log n)
sort(v.rbegin(), v.rend());          // decrescente
stable_sort(v.begin(), v.end());     // O(n log^2 n)
reverse(v.begin(), v.end());         // O(n)

// comparador: true se a vem antes de b
sort(v.begin(), v.end(),
     [](const P& a, const P& b) {
         if (a.first != b.first)
             return a.first > b.first;
         return a.second < b.second;
     });

// k-ésimo menor sem ordenar tudo - O(n) médio
nth_element(v.begin(), v.begin() + k, v.end());
// v[k] certo; menores à esquerda, maiores à direita

// só os k primeiros ordenados - O(n log k)
partial_sort(v.begin(), v.begin() + k, v.end());
```

Busca binária (vetor **ordenado**), $O(\log n)$:

```
lower_bound(v.begin(), v.end(), x); // 1º >= x
upper_bound(v.begin(), v.end(), x); // 1º >  x
binary_search(v.begin(), v.end(), x);  // bool
equal_range(v.begin(), v.end(), x);    // par

int i = lower_bound(v.begin(), v.end(), x)
      - v.begin();                  // índice
int qtd = upper_bound(...) - lower_bound(...);
```

Comprimir coordenadas ($O(n \log n)$):

```
vector<int> c = v;
sort(c.begin(), c.end());
c.erase(unique(c.begin(), c.end()), c.end());
// valor x vira o índice:
int id = lower_bound(c.begin(), c.end(), x)
       - c.begin();
```

`unique` só remove repetidos **adjacentes**, então ordene antes.

4 set, multiset e map

Árvore balanceada, sempre ordenada, $O(\log n)$ por operação.

```
set<int> s;
s.insert(x);            // par<it, bool> inserido?
s.erase(x);             // apaga todos iguais a x
s.erase(it);            // O(1) amortizado
s.count(x);             // 0 ou 1
s.find(x);              // it ou s.end()
s.lower_bound(x);       // use o do set, não o
s.upper_bound(x);       // global (este é O(log n))
*s.begin();             // menor
*s.rbegin();            // maior

map<string, int> f;
f[k]++;                 // cria com 0 se não existe
f.count(k); f.find(k);
for (auto& [k, val] : f) ...   // em ordem de k

multiset<int> ms;       // permite repetidos
ms.erase(ms.find(x));   // apaga UMA ocorrência
ms.erase(x);            // apaga TODAS
```

`lower_bound(s.begin(), s.end(), x)` global é $O(n)$ em `set`, porque o iterador não é de acesso aleatório. Sempre use `s.lower_bound(x)`.

Anterior e sucessor de `x`:

```
auto it = s.lower_bound(x);
if (it != s.end())     /* menor valor >= x */;
if (it != s.begin()) { --it; /* maior < x */ }
```

Iterador de `set`/`map` continua válido depois de inserir; só quem é apagado é invalidado.

5 unordered_set e unordered_map

Hash: $O(1)$ médio, $O(n)$ pior caso. Mesma interface, **sem** `lower_bound` e sem ordem.

```
unordered_map<ll, int> f;
f.reserve(1 << 20);
f.max_load_factor(0.25);   // menos colisões
```

Juízes com testes anti-hash podem derrubar `unordered_map<int,...>` para $O(n)$. Proteção barata:

```
struct H {
    size_t operator()(ll x) const {
        x += 0x9e3779b97f4a7c15LL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9LL;
        return x ^ (x >> 31);
    }
};
unordered_map<ll, int, H> f;
```

Na dúvida, `map` com $O(\log n)$ garantido costuma ser rápido o bastante.

6 priority_queue, queue, stack, deque

```
// máximo no topo (padrão)
priority_queue<int> pq;
// mínimo no topo
priority_queue<int, vector<int>, greater<int>> mn;

pq.push(x);   // O(log n)
pq.top();     // O(1) - não remove
pq.pop();     // O(log n) - não devolve nada
pq.size(); pq.empty();
// construir de um vetor: O(n)
priority_queue<int> p(v.begin(), v.end());

queue<int> q;                  // FIFO
q.push(x); q.front(); q.pop();

stack<int> st;                 // LIFO
st.push(x); st.top(); st.pop();

deque<int> dq;                 // duas pontas, O(1)
dq.push_front(x); dq.push_back(x);
dq.pop_front();   dq.pop_back();
dq[i];                         // acesso aleatório O(1)
```

`priority_queue` não deixa alterar prioridade: insira de novo e ignore entradas velhas (`if (d != dist[u]) continue;`).

7 Algoritmos gerais

Todos em `<algorithm>` ou `<numeric>`; `n` é o tamanho do intervalo.

```
// O(n)
*min_element(v.begin(), v.end());
*max_element(v.begin(), v.end());
minmax_element(v.begin(), v.end());
accumulate(v.begin(), v.end(), 0LL);   // soma
count(v.begin(), v.end(), x);
count_if(v.begin(), v.end(), pred);
find(v.begin(), v.end(), x);           // it/end()
all_of / any_of / none_of(b, e, pred);
fill(v.begin(), v.end(), x);
iota(v.begin(), v.end(), 0);           // 0,1,2,...
partial_sum(v.begin(), v.end(), s.begin());
adjacent_difference(b, e, d.begin());
rotate(v.begin(), v.begin() + k, v.end());
swap(a, b);
copy(v.begin(), v.end(), w.begin());
```

```cpp
// O(n log n)
sort / stable_sort
// O(n) em containers ordenados
set_union / set_intersection /
set_difference(a.begin(), a.end(),
               b.begin(), b.end(),
               back_inserter(out));
merge(a.begin(), a.end(), b.begin(), b.end(),
      out.begin());

// permutações - O(n) por chamada
do {
    // usa a permutação atual
} while (next_permutation(v.begin(), v.end()));
// percorrer todas: O(n! * n) - comece ordenado
```

`max({a, b, c})` e `min({a, b, c})` aceitam lista. `__gcd(a, b)` e `gcd/lcm` (C++17) são $O(\log \min)$.

8 string

```cpp
string s = "abc";
s += 'd'; s += "ef";      // O(1) amortizado
s.size(); s[i];           // O(1)
s.substr(pos, len);       // O(len) - copia!
s.find("ab");             // O(n*m), npos se não achar
s.rfind, s.find_first_of
s.insert(i, t); s.erase(i, len);  // O(n)
stoi(s); stoll(s); stod(s);
to_string(x);
reverse(s.begin(), s.end());
sort(s.begin(), s.end());

// dividir por espaços
stringstream ss(linha);
string tok;
while (ss >> tok) ...

// ler linha inteira (cuidado com o '\n' solto)
getline(cin, linha);
```

Comparar strings é $O(n)$, então `set<string>` custa $O(n \log k)$ por operação, não $O(\log k)$.

9 bitset

`bitset<N>` faz operações em blocos de 64 bits: $O(N/64)$ em vez de $O(N)$. Ganho de ~64x.

```cpp
bitset<1000> b;
b[i] = 1; b.set(i); b.reset(i); b.flip(i);
b.set();              // todos 1
b.count();            // popcount, O(N/64)
b.any(); b.none(); b.all();
b._Find_first();      // 1º bit 1 (N se não há)
b._Find_next(i);      // próximo bit 1 depois de i
a |= b; a &= b; a ^= b; a <<= k;
b.to_string(); b.to_ullong();
```

`N` precisa ser constante de compilação. Útil em DP de subset-sum: `dp |= dp << w;`

10 Truques de bits

```cpp
__builtin_popcountll(x);  // nº de bits 1
__builtin_clzll(x);       // zeros à esquerda
__builtin_ctzll(x);       // zeros à direita
63 - __builtin_clzll(x);  // floor(log2 x), x > 0
x & (x - 1);              // apaga o bit 1 mais baixo
x & -x;                   // isola o bit 1 mais baixo
x >> i & 1;               // lê o bit i
x |= 1LL << i;            // liga o bit i
x &= ~(1LL << i);         // desliga o bit i
```

Todos $O(1)$. As versões sem `ll` são para `int`; `__builtin_clz(0)` é indefinido.

11 Armadilhas comuns

- `1 << 31` estoura `int`: use `1LL << k`.
- `v.size()` é **sem sinal**; `v.size() - 1` com vetor vazio vira um número gigante. Compare com `(int)v.size()`.
- Iterador invalidado depois de `push_back` (realocação) ou `erase`. Em loop: `it = v.erase(it);` sem `++it`.
- `m[k]` em `map` **cria** a chave com valor 0. Para só consultar, use `m.count(k)` ou `m.find(k)`.
- `pq.pop()` não devolve o elemento; leia `pq.top()` antes.
- `s.erase(x)` em `multiset` apaga **todos** os iguais.
- Comparador de `sort` tem que ser estritamente menor: usar `<=` dá *runtime error*.
- `cin >> x` seguido de `getline` lê uma linha vazia: consuma o `\n` com `cin.ignore()`.

12 E/S rápida

```cpp
ios::sync_with_stdio(false);
cin.tie(nullptr);           // NÃO use com scanf/printf
cout << fixed << setprecision(10);
'\n' em vez de endl;        // endl dá flush toda vez
```